FF

FrameForge Studio

LOCAL VIDEO STUDIO

FrameForge Studio

Guide & Operations Manual

Version 0.1.0 · Electron desktop application · Windows, macOS, Linux

Rendered on this machine · 2026-11-10 · No backend server

Contents

01 What FrameForge Studio is 3

02 How a render is put together 3

03 System requirements 4

04 Install and first run 4

05 The interface 5

06 API keys 6

07 Making your first video 7

08 Writing the brief 8

09 Cost and timing 9

10 Files and folders 9

11 Checking your installation 10

12 Troubleshooting 11

13 Security and privacy 12

14 Distributing the application 12

15 Environment variables 13

16 Glossary 13

1 What FrameForge Studio is

FrameForge Studio is a desktop application for producing short motion-graphic videos from a written brief. You describe the piece, your own language model writes a multi-scene storyboard as animated HTML, and the app renders a finished MP4 on your machine.

Three things define the product:

1. **There is no video API and no FrameForge server.** The only network call goes to the

language model you configure, with your key. Every frame is drawn locally.

1. **You supply the API key.** One OpenAI-compatible key — DeepSeek, OpenRouter, Groq or a

local Ollama — is enough. It is stored in your operating system's credential store.

1. **Rendering happens locally.** Chromium captures each frame; FFmpeg assembles, titles

and encodes. Nothing is uploaded for rendering.

What it is not: a hosting service, a stock-footage library, a timeline editor, or a billing relationship. There is no account to create, no subscription, and no usage reporting.

2 How a render is put together

Every job runs the same seven stages, in order. The Jobs screen shows each one live.

#	STAGE	WHAT HAPPENS	TYPICAL TIME
1	Validate inputs & toolchain	Checks FFmpeg is installed, a language model is configured, and the brief is not empty	instant
2	Storyboard from your model	Your model writes a multi-scene storyboard: one self-contained animated HTML document per scene	10–40 s
3	Render scenes locally	Each scene is rendered frame by frame in Chromium, then encoded to H.264 by FFmpeg	5–20 s per scene
4	Download source clips	Skipped — scenes are rendered locally, so there is nothing to fetch	n/a
5	Compose title cards	Renders opening and closing cards, one PNG per frame	10–20 s
6	Render with FFmpeg	Normalises every segment, joins them, writes the MP4	10–60 s
7	Export deliverables	Writes the poster frame, optional `.pptx` deck, and file records	seconds

Stages 5 and 6 are the parts that always run locally, even if you skip the documents. A job is never partly uploaded: if the app closes mid-render, the job is marked interrupted on next launch and the files that already exist are left on disk.

The video is assembled as: opening card → each generated scene → closing card, joined with either hard cuts or a crossfade. Card durations are 3.2 s (2.6 s in portrait) for the opening and 2.8 s (2.2 s in portrait) for the closing.

---

3 System requirements

REQUIREMENT	DETAIL
Operating system	Windows 10/11, macOS 12+, or a modern Linux desktop
Node.js	22 or newer (only needed to build or run from source)
pnpm	10 or newer
FFmpeg	`ffmpeg` and `ffprobe` on `PATH`, or set `FRAMEFORGE_FFMPEG_DIR`
OfficeCLI	Optional; bundled as an npm dependency. Document stages are skipped without it
Disk	Roughly 40 MB per 10 s of 1080p output, plus intermediate frame PNGs
Network	Outbound HTTPS to your language-model endpoint only (none with a local Ollama)

FFmpeg 6 or newer is recommended. The build is verified against FFmpeg 9.x on Windows.

---

4 Install and first run

From source

```
pnpm install
pnpm dev
```

`pnpm install` downloads the Electron runtime and unpacks the OfficeCLI binary. If your network blocks either, see §12.

Production build

```
pnpm build          # typecheck + bundle main, preload and renderer
pnpm package        # unpacked application in release/
pnpm dist           # platform installer (NSIS / DMG / AppImage)
```

First launch

1. The window opens on **Studio**.
2. Check the status pills in the header:

- **FFmpeg** — must be green. Red means FFmpeg was not found. - **OfficeCLI ready** — amber if missing; only document stages are affected. - **OS keychain** — green when an OS-backed vault is available.

1. Open **Keys** and add your language-model key (§6).
2. Return to **Studio** and press **Generate scenes**.

---

5 The interface

The application has four sections, reachable from the header.

Studio

The authoring surface, in two columns.

- **Left — the composer.** Project title and output name, the engine (your model, the

film length, scene count and join), the brief, title-card copy and accent colour, and the Office deliverables toggles. The bar at the bottom reads `$0.00` and holds the **Generate scenes** button.

- **Right — the bookend preview.** The opening card as it will render, drawn from the

same HTML the encoder captures. Below it, the render plan and the status of the most recent job.

The preview updates about half a second after you stop typing. It previews the *opening card*; scenes do not exist until your model writes them during a run.

Jobs

The render queue on the left, the selected job on the right.

- Each row shows the title, status, progress bar, engine and age.
- The detail panel shows all seven stages with their status and a one-line result, the

artifact list with sizes, and a **Show in folder** action per artifact.

- **Cancel** stops a running job at its next checkpoint. Scenes already encoded are kept.

Jobs are stored as JSON files and survive restarts. One job runs at a time; additional jobs queue in order.

Keys

A single card — the universal language model — showing whether a key is configured, its last four characters and a short fingerprint, plus **Test**, **Remove** and the storage explanation.

System

The runtime versions, toolchain discovery results (with paths), vault backend, the exact list of hosts the app may contact, and where your workspace lives.

---

6 API keys

Where keys come from

Create a key in your model provider's own dashboard — DeepSeek, OpenRouter, Groq, or a local Ollama. FrameForge never proxies, resells or inspects your usage.

There are no video-provider keys

Runway, Luma, fal.ai, Replicate, HeyGen and OpenAI Sora are **not part of this build's workflow.** Their adapters still exist in `src/main/providers/` for a future video-backed mode, but the interface never asks for those keys and no job can require one. The film is drawn, not purchased: the only spend is your language model's tokens.

Universal LLM (DeepSeek, OpenRouter, Groq, Ollama…)

The **Custom / Universal provider** card accepts any OpenAI-compatible chat endpoint. It is the engine of the whole app. It takes four things:

FIELD	EXAMPLE
Provider name	`DeepSeek`
Base URL	`https://api.deepseek.com/v1`
Model ID	`deepseek-chat`
API key	your key for that service

Presets fill the first three for DeepSeek, OpenRouter, Groq and a local Ollama, and any other endpoint that speaks `POST /v1/chat/completions` works the same way.

This provider **is** the engine: it writes the storyboard and the animated scene documents that become the film. There is no video API in the workflow, so this is the only key the app needs. Name, base URL and model go to `settings.json`; the key goes to the OS credential store under `provider:custom`, and **Test** performs one authenticated `GET /models` so a wrong key is caught without spending tokens.

The app builds the HTTP client for this provider from your base URL alone: that single host is allowlisted for that client, so no other address becomes reachable. Loopback addresses are permitted only because you typed them (useful for Ollama or LM Studio on your own machine).

How keys are stored

- The key is written to the operating system's credential store: **Windows Credential

Manager, macOS Keychain, or Secret Service** (libsecret) on Linux. The service name is `FrameForge Studio` and the account name is `provider:<id>`.

- If the native keyring module cannot load, the app falls back to Electron `safeStorage`,

which uses DPAPI on Windows and the Keychain on macOS. On Linux it refuses the `basic_text` backend, because that is not encryption.

- Only three non-secret facts are written to disk in `vault/keys.json`: the last four

characters, the first 12 hex characters of a SHA-256 fingerprint, and a timestamp.

What the app refuses

- Keys shorter than 8 characters, longer than 4096, containing spaces, tabs or control

characters, or wrapped in quotes. Paste the raw value.

- Storing a key when no OS-backed encryption is available: the app falls back to

session-only memory and says so in the interface.

Rotating and removing

Remove deletes both the credential-store entry and the metadata. To rotate, remove then store the new key; the fingerprint and last four characters change, which is how you confirm the swap.

Test performs a single read-only request against the provider and reports latency. It treats a 401 or 403 as a rejected key and any other status as accepted — it never starts a generation, so testing costs nothing.

---

7 Making your first video

1. **Title** — internal name, shown in Jobs.
2. **Output file** — the filename stem for the MP4. Letters, numbers, spaces and dashes.
3. **Aspect** — 16:9 (1920×1080), 9:16 (1080×1920) or 1:1 (1080×1080).
4. **Engine** — *Motion scenes (free)*. The card shows which model is connected, or an

add key link if none is.

1. **Length** — total film length: 8, 10, 12, 15, 20 or 30 seconds.

2. **Scenes** — 2 to 8. The app divides the length evenly and tells the model how many

scenes to write.

1. **Join** — Crossfade or hard cut between scenes.
2. **Brief** — what the film is about. See §8.
3. **Bookend cards** — headline (80 characters), subhead (120), accent colour. Switch

Bookend cards off to render scenes only.

1. **Deliverables** — the scene list (`.json`) is written next to the film; the `.pptx`

delivery deck is generated by OfficeCLI when you switch it on.

1. Review the preview, then **Generate scenes**.

If the button is disabled, the reason is stated directly beneath it: FFmpeg missing, no language model configured, or a brief that is too short.

What you get

Written to the output folder (§10):

- `<output-name>.mp4` — H.264, `yuv420p`, 30 fps, `+faststart`, no audio track.
- `<output-name>-poster.jpg` — a frame from 1.5 s in (or mid-film for short pieces).
- `storyboard.json` — the scene list: title, duration and HTML size per scene.
- A delivery `.pptx` when requested.

---

8 Writing the brief

The brief is the whole direction, and it is the only thing the model sees besides the length and scene count. A brief that names the message, the audience, the on-screen words and the palette produces a usable film; a brief that names a concept produces wallpaper.

A workable shape:

> *[what the film is for] + [the exact words that must appear] + [the visual system] + > [the mood, and what to avoid]*

Example:

> A 15-second teaser for a Q4 product launch. Open on “Built for the next launch”, then > build momentum with Ship, Measure, Repeat. Geometric shapes locking into place, a > violet-to-cyan gradient field, heavy grotesque type. Confident and technical, no stock > imagery, no people.

Practical notes:

- **Write the on-screen copy yourself.** Kinetic typography is only as good as the words

in it; the model will invent filler unless you supply the lines.

- **Name the palette.** Otherwise you get whatever the model defaults to, and scenes will

not feel like one film.

- **One idea per scene.** At 3–5 seconds a scene holds one line, not a paragraph.
- **Say what to avoid.** “No stock photos, no clip-art icons, no lens flares” is as useful

as a positive instruction.

- **Keep it under ~120 words.** Beyond that the model starts summarising instead of

following.

Each scene comes back as a complete HTML document with its own CSS animation. The app sanitises it (see §13), drives the animation frame by frame, and encodes the result. If a scene cannot be used, a branded fallback card stands in for it so the film still finishes.

---

9 Cost and timing

The action bar reads `$0.00` because there is nothing to buy: no video API is ever called. The only cost is language-model tokens for the storyboard — a few thousand per job, which is fractions of a cent on DeepSeek and nothing at all on a local Ollama.

STEP	TYPICAL TIME (12 S FILM, 3 SCENES)
Storyboard from your model	5–25 s
Scene capture + encode, per scene	4–10 s
Bookend cards	6–12 s
Join and export	4–10 s

Capture is wall-clock bound: every frame is a real Chromium paint at 30 fps, so a 30-second 1080p film is 900 frames per pass and takes several minutes on a laptop. If a render feels slow, cut the length or the scene count before changing anything else.

Duration handling: the model proposes a duration per scene and the app rescales the set so the film matches the length you chose, within a second.

---

10 Files and folders

Everything lives under the application's data directory:

PLATFORM	DATA DIRECTORY
Windows	`%APPDATA%\frameforge-studio`
macOS	`~/Library/Application Support/frameforge-studio`
Linux	`~/.config/frameforge-studio`

Inside it:

PATH	CONTENTS
`workspace/output/`	Final MP4s and posters — this is the folder you deliver from
`workspace/projects/<job-id>/clips/`	Encoded scenes and the encoded cards
`workspace/projects/<job-id>/frames/`	Per-frame PNGs for the title cards
`workspace/projects/<job-id>/docs/`	Storyboard `.docx`, delivery `.pptx`
`jobs/<job-id>.json`	Job records, including stage history and artifact paths
`vault/keys.json`	Key metadata only — never secret material

Frame PNGs are kept so a render can be reproduced frame-for-frame; delete the project folder when a job is finished if disk space matters.

---

11 Checking your installation

Four commands, in increasing scope:

```
pnpm typecheck      # strict TypeScript, no `any`
pnpm test           # 72 tests: vault, scenes, providers, LLM, FFmpeg, pipeline, jobs,
documents
pnpm smoke          # boots the real app, checks the bridge and vault, writes smoke.png
pnpm smoke:job      # full free render through the UI against a local mock model
```

`smoke:job` needs no account. It points the app at a local mock that speaks the OpenAI shape, drives a three-scene job through the real interface, and verifies the exported file with ffprobe. Use it after any change to confirm the pipeline still works end to end.

`smoke` also round-trips a throwaway key through the real credential store and removes it again, so a passing run proves persistence, not just the IPC path.

---

12 Troubleshooting

SYMPTOM	CAUSE	FIX
Header pill shows **FFmpeg missing**; render button disabled	`ffmpeg` / `ffprobe` not on `PATH`	Install FFmpeg, or set `FRAMEFORGE_FFMPEG_DIR` to the folder containing both binaries, then restart
Stage 1 fails with "Add your DeepSeek … key under Keys"	No language model configured	Add the key in **Keys**, press **Test**, then re-run
Test reports "Provider rejected the key (401)"	Wrong key, revoked key, or a base URL with the wrong path	Check the base URL ends in `/v1`; regenerate the key
Storyboard fails with "The model did not return valid JSON"	The endpoint answered with prose, or the model is too small to follow the scene schema	Retry; use a larger model (`deepseek-chat` , `llama-3.3-70b`)
A scene renders as a plain card but the job succeeds	That scene's HTML failed validation, so the branded fallback was used	The generate stage detail reports the fallback count
Capture is very slow	Offscreen Chromium painting at 30 fps on a busy machine	Cut the length or scene count; close other GPU-heavy apps
Preview panel is dark	Composition not yet built, or the frame was never measured	Wait a second; re-open Studio. If it persists, run `pnpm smoke` and inspect the capture
Job fails with `EPERM … rename` on Windows	A file in the workspace is locked by another program	The app retries automatically; if it still fails, close indexers/antivirus scans on the data directory
Only document stage fails	OfficeCLI exited non-zero	Run `officecli --version` and `officecli whoami` in a terminal. First use needs a network download of its binary
Credential store unavailable; keys vanish on restart	No keyring on the machine (common on bare Linux)	Install libsecret / a keyring daemon, or accept session-only storage
OfficeCLI fails to unpack on Windows from git-bash	MSYS `tar` treats `C:` as a remote host	Start the app from a normal shell, or run `officecli --version` once from PowerShell to unpack the binary
`pnpm install` did not download Electron	Network/proxy blocked the binary download	Set the standard Electron mirror environment variables and reinstall

When a job fails, the failing stage carries the original message — your model's error text, or FFmpeg's. That text is the most precise diagnostic available — read it before changing anything.

13 Security and privacy

Keys. Stored in the OS credential store. Never sent anywhere except the provider that owns them, in an `Authorization` header from the main process. They are never returned to the interface — the renderer can set, test and delete a key, but cannot read it back.

Network. The renderer cannot make arbitrary requests. In this build the only outbound call is the one to your language-model host: the API client is built from your base URL, so exactly that host is allowlisted, and only for that client. Loopback is permitted only because you typed it (Ollama, LM Studio).

Model output is untrusted. Scene HTML arrives from a third party and is handled as data, never as code. Before capture the app removes every `<script>`, `<iframe>`, `<object>`, `<embed>`, `<link>`, `<form>` and `<base>` element, strips inline `on*` handlers, and neutralises remote `src` / `href` / `url()` / `@import` references. It then injects its own timeline driver (pause + seek) so frames are captured deterministically. Capture runs under `default-src 'none'`, so even a reference the sanitiser missed cannot reach the network.

The window. The renderer runs sandboxed with context isolation, no Node integration, a strict Content-Security-Policy (`default-src 'none'`), denied navigation, denied pop-ups, and denied permission requests other than clipboard writes.

Child processes. FFmpeg and OfficeCLI are spawned with argument arrays, never through a shell, so a file name or prompt cannot become a command.

Files. Writes are atomic. Job records are reconciled on launch, so a crash cannot leave a job stuck in "running".

Telemetry. None. There is no analytics, crash reporting or update ping.

---

14 Distributing the application

```
pnpm package    # unpacked app for smoke-testing
pnpm dist       # platform installers
```

`@napi-rs/keyring` and `officecli` are unpacked from the asar archive because they contain native executables; this is configured in `package.json` under `build.asarUnpack`. Installers are unsigned by default — signing and notarisation belong in your release pipeline.

---

15 Environment variables

VARIABLE	PURPOSE
`FRAMEFORGE_FFMPEG_DIR`	Folder containing `ffmpeg` and `ffprobe`
`FFMPEG_DIR`	Fallback for the same
`FRAMEFORGE_OFFICECLI`	Explicit path to the `officecli` binary
`FRAMEFORGE_DEV_PROVIDER`	Provider id to redirect (testing)
`FRAMEFORGE_DEV_BASE_URL`	Base URL for that provider, e.g. a local gateway or mock
`FRAMEFORGE_SMOKE`	Enable the self-check harness
`FRAMEFORGE_SMOKE_OUT`	Where the self-check writes its screenshot
`FRAMEFORGE_SMOKE_TAB`	Section to open before capturing (`keys` , `jobs` , `system`)
`FRAMEFORGE_SMOKE_JOB`	Run a full job through the UI during the self-check
`FRAMEFORGE_SMOKE_LLM_URL`	Point the self-check at an OpenAI-compatible mock, so the storyboard and prompt-enhancement paths are exercised too

`FRAMEFORGE_DEV_*` exist so the desktop app can be pointed at a mock or a self-hosted gateway during testing. Leave them unset in normal use — they also relax the loopback rule for that one provider.

16 Glossary

Aspect — output frame shape: 16:9, 9:16 or 1:1. **Artifact** — any file a job produced: clip, composition, video, poster, storyboard, deck. **Composition** — the HTML/CSS title card, animated on a paused timeline and captured one frame at a time. **Job** — one complete run of the seven stages, identified by a UUID. **Shot** — one scene of the film: an HTML document the model wrote, captured as frames. **Stage** — one step of a job, with its own status and message. **Vault** — the OS credential store holding provider keys, plus the metadata file.

FrameForge Studio 0.1.0 — Guide & Operations Manual